

Image Forgery Detection

Generated by Doxygen 1.17.0

1 Görüntü Sahteciliği Tespit Sistemi API Dökümantasyonu	1
1.1 Giriş	1
2 Görüntü Sahteciliği Tespit Sistemi (Image Forgery Detection)	3
2.1 Projenin Amacı	3
2.2 Teknik Çalışma Prensibi	3
2.2.1 1. Geleneksel Bilgisayarlı Görü Analizi	3
2.2.1.1 Nasıl çalışır?	4
2.2.1.2 1. Anahtar Nokta Tespiti (Keypoints)	4
2.2.1.3 2. Descriptor (Betimleyici) Oluşturma	4
2.2.1.4 3. Eşleşme (Matching)	4
2.2.1.5 4. Mekansal Filtreleme	4
2.2.1.6 5. Geometrik Tutarlılık Analizi	4
2.2.1.7 Sonuç Mantığı	4
2.2.1.8 Görselleştirme	4
2.2.2 2. Yapay Zeka Analizi (CNN + LSTM)	5
2.2.2.1 Kullanılan Model:	5
2.2.2.2 Dataset Bilgisi	5
2.2.2.3 Çalışma Adımları	5
2.2.2.4 AI Skoru	5
2.3 Hibrit Karar Mekanizması	6
2.3.1 Proje Mimarisi	6
2.3.2 Kurulum ve Çalıştırma	6
2.3.3 Geliştirici Notu	6
3 Directory Hierarchy	7
3.1 Directories	7
4 Namespace Index	9
4.1 Package List	9
5 Hierarchical Index	11
5.1 Class Hierarchy	11
6 Class Index	13
6.1 Class List	13
7 File Index	15
7.1 File List	15
8 Directory Documentation	17
8.1 forgery-ui Directory Reference	17
8.1.1 Detailed Description	17
8.1.2 Getting Started with Create React App	17
8.1.3 Available Scripts	17

8.1.3.1 <code>npm start</code>	17
8.1.3.2 <code>npm test</code>	17
8.1.3.3 <code>npm run build</code>	17
8.1.3.4 <code>npm run eject</code>	17
8.1.4 Learn More	17
8.1.4.1 Code Splitting	18
8.1.4.2 Analyzing the Bundle Size	18
8.1.4.3 Making a Progressive Web App	18
8.1.4.4 Advanced Configuration	18
8.1.4.5 Deployment	18
8.1.4.6 <code>npm run build</code> fails to minify	18
9 Namespace Documentation	19
9.1 <code>ai_analysis</code> Namespace Reference	19
9.1.1 Function Documentation	19
9.1.1.1 <code>run_ai_analysis()</code>	19
9.1.2 Variable Documentation	19
9.1.2.1 <code>cnx_model</code>	19
9.1.2.2 <code>device</code>	19
9.1.2.3 <code>fc</code>	20
9.1.2.4 <code>lstm_model</code>	20
9.1.2.5 <code>model_path</code>	20
9.1.2.6 <code>resnet</code>	20
9.1.2.7 <code>transform</code>	20
9.2 Main Namespace Reference	20
9.2.1 Function Documentation	20
9.2.1.1 <code>analyze_image()</code>	20
9.2.2 Variable Documentation	20
9.2.2.1 <code>allow_headers</code>	20
9.2.2.2 <code>allow_methods</code>	20
9.2.2.3 <code>allow_origins</code>	21
9.2.2.4 <code>app</code>	21
9.2.2.5 <code>host</code>	21
9.2.2.6 <code>port</code>	21
9.3 <code>traditional_analysis</code> Namespace Reference	21
9.3.1 Function Documentation	21
9.3.1.1 <code>draw_matches()</code>	21
9.3.1.2 <code>find_forgery_matches()</code>	21
9.3.1.3 <code>get_detector()</code>	22
9.3.1.4 <code>run_traditional_analysis()</code>	22
9.4 <code>train_model</code> Namespace Reference	23
9.4.1 Variable Documentation	23

9.4.1.1 criterion	23
9.4.1.2 device	23
9.4.1.3 EPOCHS	23
9.4.1.4 fc	24
9.4.1.5 images	24
9.4.1.6 labels	24
9.4.1.7 loss	24
9.4.1.8 model	24
9.4.1.9 optimizer	24
9.4.1.10 outputs	24
9.4.1.11 running_loss	24
9.4.1.12 test_dataset	24
9.4.1.13 test_loader	24
9.4.1.14 train_dataset	24
9.4.1.15 train_loader	24
9.4.1.16 transform	25
10 Class Documentation	27
10.1 ai_analysis.ForgeryLSTM Class Reference	27
10.1.1 Detailed Description	28
10.1.2 Constructor & Destructor Documentation	28
10.1.2.1 __init__()	28
10.1.3 Member Function Documentation	28
10.1.3.1 forward()	28
10.1.4 Member Data Documentation	29
10.1.4.1 fc	29
10.1.4.2 lstm	29
11 File Documentation	31
11.1 ai_analysis.py File Reference	31
11.2 forgery-ui/README.md File Reference	31
11.3 Readme.md File Reference	31
11.4 Main.py File Reference	31
11.5 traditional_analysis.py File Reference	32
11.6 train_model.py File Reference	32
Index	33

Chapter 1

Görüntü Sahteciliği Tespit Sistemi API Dökümantasyonu

1.1 Giriş

Bu proje, dijital görüntüler üzerindeki manipülasyonları ve Copy-Move sahteciliklerini geleneksel bilgisayarlı görü ve derin öğrenme hibrit mimarisiyle tespit eder.

Chapter 2

Görüntü Sahteciliği Tespit Sistemi (Image Forgery Detection)

Bu proje, bir dijital görüntünün manipüle edilip edilmediğini hem **Geleneksel Bilgisayarlı Görü (Computer Vision)** hem de **Derin Öğrenme (AI)** tekniklerini kullanarak analiz eden bir web uygulamasıdır.

Sistem özellikle **Copy-Move (Kopyala-Yapıştır)** sahteciliğine aday bölgeleri tespit etmeye odaklanır.

2.1 Projenin Amacı

Bu sistem:

- Görüntü içerisindeki **tekrar eden yapısal bölgeleri**
- Olası **kopyalanmış (copy-move) alanları**
- Ve piksel seviyesinde **anormallikleri**

tespit ederek kullanıcıya **şüpheli bölgeleri görsel olarak sunar**.

Not: Sistem **kesin sahtecilik kararı vermez**, bunun yerine **şüpheli bölgeleri (candidate regions)** işaretler.

2.2 Teknik Çalışma Prensibi

Sistem iki farklı analiz katmanından oluşur:

2.2.1 1. Geleneksel Bilgisayarlı Görü Analizi

Kullanılan algoritmalar:

- SIFT
- SURF
- AKAZE
- ORB

Bu algoritmalar Computer Vision alanında kullanılan **feature-based yöntemlerdir**.

2.2.1.1 Nasıl çalışır?

2.2.1.2 1. Anahtar Nokta Tespiti (Keypoints)

Görüntüdeki karakteristik bölgeler bulunur:

- köşeler
- kenarlar
- texture değişimleri

2.2.1.3 2. Descriptor (Betimleyici) Oluşturma

Her nokta için matematiksel bir "kimlik" çıkarılır.

2.2.1.4 3. Eşleşme (Matching)

Aynı görüntü içindeki noktalar karşılaştırılır:

- KNN (k=2) kullanılır
- Lowe's Ratio Test ile zayıf eşleşmeler elenir

2.2.1.5 4. Mekansal Filtreleme

- Noktalar birbirine çok yakınsa elenir
- Minimum mesafe: **50 piksel**

2.2.1.6 5. Geometrik Tutarlılık Analizi

En kritik adım:

- Eşleşmelerin vektörleri hesaplanır
- Median vektör bulunur
- Sadece bu vektöre yakın olanlar seçilir

Bu, copy-move sahteciliğinin temel özelliğidir:

"Tüm kopyalanan bölge aynı yönde ve aynı miktarda taşınır"

2.2.1.7 Sonuç Mantiğı

- Eğer **10'dan fazla tutarlı eşleşme** varsa:

Görüntü "**Şüpheli**" olarak işaretlenir

2.2.1.8 Görselleştirme

Sistem:

- eşleşen noktaları
- bağlantı çizgilerini
- kaynak ve hedef bölgeleri

görsel olarak işaretler:

- kaynak noktalar
- hedef noktalar
- eşleşme çizgileri
- / bounding box alanları

2.2.2 2. Yapay Zeka Analizi (CNN + LSTM)

Bu katman:

- insan gözünün fark edemediği
- piksel seviyesindeki anomalileri

tespit etmeye çalışır.

2.2.2.1 Kullanılan Model:

- ResNet-18 (CNN)
 - LSTM (sequence modeling)
-

2.2.2.2 Dataset Bilgisi

AI modeli:

- Deepfake (yüz manipülasyonu) dataseti kullanılarak eğitilmiştir.
- Dataset **Kaggle** platformundan alınmıştır
- Dataset Link: <https://www.kaggle.com/datasets/underscore/fake-vs-real-face-classification>

Önemli Not:

Bu dataset:

- Yüz sahteciliği (deepfake) odaklıdır.
- Copy-move sahteciliği için özel olarak üretilmemiştir.

Bu nedenle AI modeli:

- Copy-move tespitinde destekleyici sinyal üretir.
 - Ancak tek başına kesin karar vermez.
-

2.2.2.3 Çalışma Adımları

1. Görüntü normalize edilir (224x224)
 2. CNN ile feature çıkarılır
 3. Feature vektörü LSTM'e verilir
 4. CNN + LSTM skorları birleştirilir
-

2.2.2.4 AI Skoru

- 0 → Temiz
 - 1 → Sahte
- %50 üzeri → Şüpheli kabul edilir
-

2.3 Hibrit Karar Mekanizması

Final karar:

$\text{Şüpheli} = (\text{Traditional}) \text{ OR } (\text{AI})$

2.3.1 Proje Mimarisi

Proje, yazılım mühendisliği prensiplerine uygun olarak modüler bir yapıda geliştirilmiştir:

```
Image Forgery Detection
main.py
traditional_analysis.py
ai_analysis.py
train_model.py
forgery-ui/
```

main.py: FastAPI tabanlı backend sunucusudur. İstekleri karşılar ve analiz sürecini yönetir.

traditional_analysis.py: SIFT, SURF, AKAZE ve ORB gibi OpenCV tabanlı algoritmaların motorudur.

ai_analysis.py: PyTorch kullanarak ResNet18 (CNN) ve LSTM modelleri ile derin analiz yapar.

train_model.py: ResNet model eğitimi yapar.

forgery-ui/: Kullanıcı dostu React tabanlı arayüzdür.

2.3.2 Kurulum ve Çalıştırma

Gereksinimler:

- Python 3.9+
- Node.js.

Backend:

```
pip install -r requirements.txt
python main.py
```

Frontend:

```
cd forgery-ui
npm install
npm start
```

2.3.3 Geliştirici Notu

Bu proje:

- eğitim amaçlıdır
- explainable AI yaklaşımı içerir
- görsel analiz sistemlerinin temelini öğretir

Chapter 3

Directory Hierarchy

3.1 Directories

forgery-ui	17
----------------------	----

Chapter 4

Namespace Index

4.1 Package List

Here are the packages with brief descriptions (if available):

ai_analysis	19
Main	20
traditional_analysis	21
train_model	23

Chapter 5

Hierarchical Index

5.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

nn.Module	
ai_analysis.ForgeryLSTM	27

Chapter 6

Class Index

6.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

[ai_analysis.ForgeryLSTM](#) 27

Chapter 7

File Index

7.1 File List

Here is a list of all files with brief descriptions:

ai_analysis.py	31
Main.py	31
traditional_analysis.py	32
train_model.py	32

Chapter 8

Directory Documentation

8.1 forgery-ui Directory Reference

8.1.1 Detailed Description

8.1.2 Getting Started with Create React App

This project was bootstrapped with [Create React App](#).

8.1.3 Available Scripts

In the project directory, you can run:

8.1.3.1 `npm start`

Runs the app in the development mode.\ Open <http://localhost:3000> to view it in your browser. The page will reload when you make changes.\ You may also see any lint errors in the console.

8.1.3.2 `npm test`

Launches the test runner in the interactive watch mode.\ See the section about [running tests](#) for more information.

8.1.3.3 `npm run build`

Builds the app for production to the `build` folder.\ It correctly bundles React in production mode and optimizes the build for the best performance.

The build is minified and the filenames include the hashes.\ Your app is ready to be deployed!

See the section about [deployment](#) for more information.

8.1.3.4 `npm run eject`

Note: this is a one-way operation. Once you `eject`, you can't go back!

If you aren't satisfied with the build tool and configuration choices, you can `eject` at any time. This command will remove the single build dependency from your project.

Instead, it will copy all the configuration files and the transitive dependencies (webpack, Babel, ESLint, etc) right into your project so you have full control over them. All of the commands except `eject` will still work, but they will point to the copied scripts so you can tweak them. At this point you're on your own.

You don't have to ever use `eject`. The curated feature set is suitable for small and middle deployments, and you shouldn't feel obligated to use this feature. However we understand that this tool wouldn't be useful if you couldn't customize it when you are ready for it.

8.1.4 Learn More

You can learn more in the [Create React App documentation](#).

To learn React, check out the [React documentation](#).

8.1.4.1 Code Splitting

This section has moved here: <https://facebook.github.io/create-react-app/docs/code-splitting>

8.1.4.2 Analyzing the Bundle Size

This section has moved here: <https://facebook.github.io/create-react-app/docs/analyzing-the-bundle-size>

8.1.4.3 Making a Progressive Web App

This section has moved here: <https://facebook.github.io/create-react-app/docs/making-a-progressive-web-app>

8.1.4.4 Advanced Configuration

This section has moved here: <https://facebook.github.io/create-react-app/docs/advanced-configuration>

8.1.4.5 Deployment

This section has moved here: <https://facebook.github.io/create-react-app/docs/deployment>

8.1.4.6 `npm run build` fails to minify

This section has moved here: <https://facebook.github.io/create-react-app/docs/troubleshooting#npm-run-build-fails-to-minify>

Chapter 9

Namespace Documentation

9.1 ai_analysis Namespace Reference

Classes

- class [ForgeryLSTM](#)

Functions

- [run_ai_analysis](#) ([img_pil](#))

Variables

- [device](#) = torch.device("cuda" if torch.cuda.is_available() else "cpu")
- [resnet](#) = models.resnet18()
- [fc](#)
- str [model_path](#) = "forgery_model.pth"
- [cnn_model](#) = nn.Sequential(*(list(resnet.children())[:-1]))
- [lstm_model](#) = [ForgeryLSTM](#)() .to([device](#))
- [transform](#)

9.1.1 Function Documentation

9.1.1.1 run_ai_analysis()

```
run_ai_analysis (  
    img_pil  
  
    @brief Derin öğrenme (CNN + LSTM) hibrit mimarisi ile görselin manipülasyon skorunu hesaplar.  
    @details PIL formatında gelen görseli ResNet standartlarına göre normalize eder, ardından  
        eğitilmiş ağırlıklar üzerinden olasılık değerlerini ve ardışık katmanlardan LSTM  
        özniteliklerini çıkararak füzyon bir güven skoru üretir.  
    @param img_pil PIL kütüphanesi formatında okunmuş RGB giriş görseli.  
    @return final_score İki modelin ortak kararı olan nihai anomali skoru yüzdesi.  
    @return status Skorun %50 eşik değerini aşıp aşmadığını belirten boolean durum bilgisi.
```

9.1.2 Variable Documentation

9.1.2.1 cnn_model

```
ai_analysis.cnn_model = nn.Sequential(*(list(resnet.children())[:-1]))
```

9.1.2.2 device

```
ai_analysis.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

9.1.2.3 fc

```
ai_analysis.fc
```

9.1.2.4 lstm_model

```
ai_analysis.lstm_model = ForgeryLSTM().to(device)
```

9.1.2.5 model_path

```
str ai_analysis.model_path = "forgery_model.pth"
```

9.1.2.6 resnet

```
ai_analysis.resnet = models.resnet18()
```

9.1.2.7 transform

```
ai_analysis.transform
```

Initial value:

```
= transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    # ResNet standart normalizasyonu
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

9.2 Main Namespace Reference

Functions

- [analyze_image](#) (UploadFile file=File(...), str algorithm=Form("SIFT"))

Variables

- [app](#) = FastAPI(title="Image Forgery Detection API", version="1.0.0")
- [allow_origins](#)
- [allow_methods](#)
- [allow_headers](#)
- [host](#)
- [port](#)

9.2.1 Function Documentation

9.2.1.1 analyze_image()

```
analyze_image (
    UploadFile file = File(...),
    str algorithm = Form("SIFT"))
```

9.2.2 Variable Documentation

9.2.2.1 allow_headers

```
Main.allow_headers
```

9.2.2.2 allow_methods

```
Main.allow_methods
```

9.2.2.3 allow_origins

```
Main.allow_origins
```

9.2.2.4 app

```
Main.app = FastAPI(title="Image Forgery Detection API", version="1.0.0")
```

9.2.2.5 host

```
Main.host
```

9.2.2.6 port

```
Main.port
```

9.3 traditional_analysis Namespace Reference

Functions

- [get_detector](#) (algo_name)
- [find_forgery_matches](#) (kp, desc, algorithm)
- [run_traditional_analysis](#) (img_cv, algorithm)
- [draw_matches](#) (img, matches)

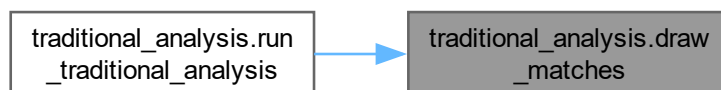
9.3.1 Function Documentation

9.3.1.1 draw_matches()

```
draw_matches (
    img,
    matches)
```

@brief Tespit edilen manipölasyon kanıtlarını görselleştirerek kullanıcı ekranına hazır hale getirir.
 @details Eşleşen kaynak ve hedef pikseller arasına yeşil adli bilişim çizgileri çizer. Sınır belirleme algoritması kullanılır.
 @param img Üzerine çizim yapılacak ham kaynak görüntü.
 @param matches Çizilecek doğrulanmış nokta çiftlerinin koordinat listesi.
 @return base64_str Web arayüzünde doğrudan '' şeklinde gösterilmeye hazır base64 stringi.

Here is the caller graph for this function:



9.3.1.2 find_forgery_matches()

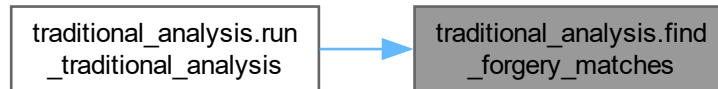
```
find_forgery_matches (
    kp,
    desc,
    algorithm)
```

```

@brief Anahtar noktalar ve betimleyiciler üzerinden kopyala-yapıştır sahteciliği araması yapar.
@details Lowe's Ratio Test ve geometrik vektör kümeleme kullanarak doğal benzerlikleri eler.
@param kp Görüntüden elde edilen anahtar noktalar (Keypoints).
@param desc Anahtar noktalara ait betimleyiciler (Descriptors).
@param algorithm Kullanılan algoritma türü.
@return Eşleşen şüpheli noktaların çiftleri ve ortalama taşıma vektörü.

```

Here is the caller graph for this function:



9.3.1.3 get_detector()

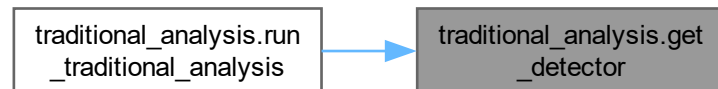
```

get_detector (
    algo_name)

@brief Kullanıcının seçtiği algoritmaya göre ilgili OpenCV nesnesini başlatır.
@param algo_name Analiz için seçilen algoritma adı (SIFT, SURF, AKAZE, ORB).
@return Başlatılan OpenCV dedektör nesnesi veya hata durumunda None.

```

Here is the caller graph for this function:



9.3.1.4 run_traditional_analysis()

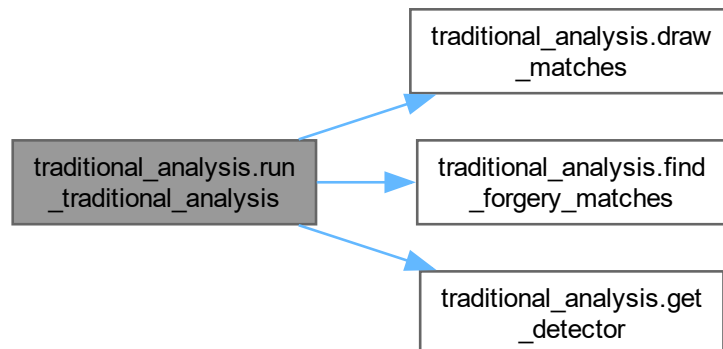
```

run_traditional_analysis (
    img_cv,
    algorithm)

@brief Geleneksel bilgisayarlı görü analiz sürecini baştan sona yöneten ana modül fonksiyonudur.
@details Görseli gri tona dönüştürür, öznetelik tespiti yapar, içsel eşleştirme fonksiyonunu tetikler ve eğer
@param img_cv OpenCV formatındaki kaynak BGR görüntü matrisi.
@param algorithm Kullanılacak algoritma türü ("SIFT", "SURF", vb.).
@return len(kp) Tespit edilen toplam anahtar nokta sayısı.
@return len(matches) Doğrulanmış sahtecilik eşleşme çizgi sayısı.
@return is_forged Görüntünün sahte olup olmadığını belirten boolean bayrak.
@return vis_image Base64 formatına kodlanmış, üzerinde kanıt çizgileri olan jpg resim string'i.

```

Here is the call graph for this function:



9.4 train_model Namespace Reference

Variables

- `device` = `torch.device("cuda" if torch.cuda.is_available() else "cpu")`
 - `transform`
 - `train_dataset`
 - `test_dataset`
 - `train_loader` = `DataLoader(train_dataset, batch_size=16, shuffle=True)`
 - `test_loader` = `DataLoader(test_dataset, batch_size=16)`
 - `model` = `models.resnet18(weights=models.ResNet18_Weights.DEFAULT)`
 - `fc`
 - `criterion` = `nn.CrossEntropyLoss()`
 - `optimizer` = `torch.optim.Adam(model.parameters(), lr=0.001)`
 - `int EPOCHS` = 3
 - `int running_loss` = 0
- Model Eğitim Döngüsü*
- `images` = `images.to(device)`
 - `labels` = `labels.to(device)`
 - `outputs` = `model(images)`
 - `loss` = `criterion(outputs, labels)`

9.4.1 Variable Documentation

9.4.1.1 criterion

```
train_model.criterion = nn.CrossEntropyLoss()
```

9.4.1.2 device

```
train_model.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

9.4.1.3 EPOCHS

```
int train_model.EPOCHS = 3
```

9.4.1.4 fc

```
train_model.fc
```

9.4.1.5 images

```
train_model.images = images.to(device)
```

9.4.1.6 labels

```
train_model.labels = labels.to(device)
```

9.4.1.7 loss

```
train_model.loss = criterion(outputs, labels)
```

9.4.1.8 model

```
train_model.model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
```

9.4.1.9 optimizer

```
train_model.optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

9.4.1.10 outputs

```
train_model.outputs = model(images)
```

9.4.1.11 running_loss

```
int train_model.running_loss = 0
```

Model Eğitim Döngüsü

Belirlenen EPOCHS değeri kadar tüm veri setini ileri ve geri besleme (Backpropagation) yöntemleriyle tarayarak `forger_model.pth` dosyasını üretir.

9.4.1.12 test_dataset

```
train_model.test_dataset
```

Initial value:

```
= datasets.ImageFolder(  
    root="dataset/test",  
    transform=transform  
)
```

9.4.1.13 test_loader

```
train_model.test_loader = DataLoader(test_dataset, batch_size=16)
```

9.4.1.14 train_dataset

```
train_model.train_dataset
```

Initial value:

```
= datasets.ImageFolder(  
    root="dataset/train",  
    transform=transform  
)
```

9.4.1.15 train_loader

```
train_model.train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
```

9.4.1.16 transform

train_model.transform

Initial value:

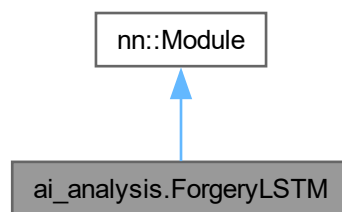
```
= transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]) #normalizasyon
])
```


Chapter 10

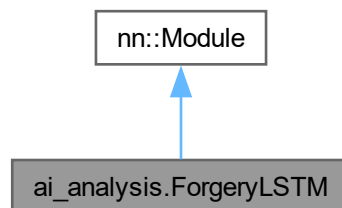
Class Documentation

10.1 ai_analysis.ForgeryLSTM Class Reference

Inheritance diagram for ai_analysis.ForgeryLSTM:



Collaboration diagram for ai_analysis.ForgeryLSTM:



Public Member Functions

- `__init__` (self)
- `forward` (self, x)

Public Attributes

- `lstm` = nn.LSTM(input_size=512, hidden_size=128, batch_first=True)

- `fc = nn.Linear(128, 1)`

10.1.1 Detailed Description

`@brief` CNN Öznitelik haritalarını ardışık veri olarak inceleyen LSTM yapay sinir ağı sınıfı.
`@details` ResNet mimarisinin son katmanından elde edilen 512 boyutlu uzaysal vektör serilerini zaman/sıra düzleminde işleyerek pikseller arası mikroskobik manipülasyonları ayırıştırır.

10.1.2 Constructor & Destructor Documentation

10.1.2.1 `__init__()`

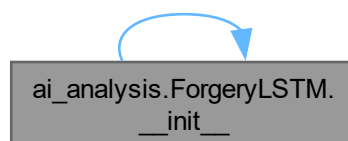
```
ai_analysis.ForgeryLSTM.__init__ (
    self)
```

`@brief` Katman yapılandırmalarını gerçekleştiren kurucu fonksiyon.
`@details` 512 girdili, 128 gizli hücreli bir LSTM katmanı ve olasılık çıktısı üreten doğrusal bir sınıflandırıcı.

Here is the call graph for this function:



Here is the caller graph for this function:



10.1.3 Member Function Documentation

10.1.3.1 `forward()`

```
ai_analysis.ForgeryLSTM.forward (
    self,
    x)
```

`@brief` Modelin ileri besleme (forward propagation) hesaplamalarını yürütür.
`@param` x CNN modelinden sıkıştırılarak aktarılan tensör verisi.
`@return` Olasılık değeri döndüren Sigmoid aktivasyon fonksiyonu çıktısı ([0, 1] arası).

10.1.4 Member Data Documentation

10.1.4.1 fc

```
ai_analysis.ForgeryLSTM.fc = nn.Linear(128, 1)
```

10.1.4.2 lstm

```
ai_analysis.ForgeryLSTM.lstm = nn.LSTM(input_size=512, hidden_size=128, batch_first=True)
```

The documentation for this class was generated from the following file:

- [ai_analysis.py](#)

Chapter 11

File Documentation

11.1 ai_analysis.py File Reference

Classes

- class [ai_analysis.ForgeryLSTM](#)

Namespaces

- namespace [ai_analysis](#)

Functions

- [ai_analysis.run_ai_analysis](#) (img_pil)

Variables

- [ai_analysis.device](#) = torch.device("cuda" if torch.cuda.is_available() else "cpu")
- [ai_analysis.resnet](#) = models.resnet18()
- [ai_analysis.fc](#)
- str [ai_analysis.model_path](#) = "forgery_model.pth"
- [ai_analysis.cnn_model](#) = nn.Sequential(*(list(resnet.children())[:-1]))
- [ai_analysis.lstm_model](#) = [ForgeryLSTM](#)()[.to\(device\)](#)
- [ai_analysis.transform](#)

11.2 forgery-ui/README.md File Reference

11.3 Readme.md File Reference

11.4 Main.py File Reference

Namespaces

- namespace [Main](#)

Functions

- [Main.analyze_image](#) (UploadFile file=File(...), str algorithm=Form("SIFT"))

Variables

- [Main.app](#) = FastAPI(title="Image Forgery Detection API", version="1.0.0")
- [Main.allow_origins](#)
- [Main.allow_methods](#)

- [Main.allow_headers](#)
- [Main.host](#)
- [Main.port](#)

11.5 traditional_analysis.py File Reference

Namespaces

- namespace [traditional_analysis](#)

Functions

- [traditional_analysis.get_detector](#) (algo_name)
- [traditional_analysis.find_forgery_matches](#) (kp, desc, algorithm)
- [traditional_analysis.run_traditional_analysis](#) (img_cv, algorithm)
- [traditional_analysis.draw_matches](#) (img, matches)

11.6 train_model.py File Reference

Namespaces

- namespace [train_model](#)

Variables

- [train_model.device](#) = torch.device("cuda" if torch.cuda.is_available() else "cpu")
- [train_model.transform](#)
- [train_model.train_dataset](#)
- [train_model.test_dataset](#)
- [train_model.train_loader](#) = DataLoader([train_dataset](#), batch_size=16, shuffle=True)
- [train_model.test_loader](#) = DataLoader([test_dataset](#), batch_size=16)
- [train_model.model](#) = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
- [train_model.fc](#)
- [train_model.criterion](#) = nn.CrossEntropyLoss()
- [train_model.optimizer](#) = torch.optim.Adam(model.parameters(), lr=0.001)
- int [train_model.EPOCHS](#) = 3
- int [train_model.running_loss](#) = 0

Model Eğitim Döngüsü

- [train_model.images](#) = images.to([device](#))
- [train_model.labels](#) = labels.to([device](#))
- [train_model.outputs](#) = model([images](#))
- [train_model.loss](#) = criterion([outputs](#), [labels](#))

Index

- `__init__`
 - `ai_analysis.ForgeryLSTM`, 28
- `ai_analysis`, 19
 - `cnn_model`, 19
 - `device`, 19
 - `fc`, 19
 - `lstm_model`, 20
 - `model_path`, 20
 - `resnet`, 20
 - `run_ai_analysis`, 19
 - `transform`, 20
- `ai_analysis.ForgeryLSTM`, 27
 - `__init__`, 28
 - `fc`, 29
 - `forward`, 28
 - `lstm`, 29
- `ai_analysis.py`, 31
- `allow_headers`
 - `Main`, 20
- `allow_methods`
 - `Main`, 20
- `allow_origins`
 - `Main`, 20
- `analyze_image`
 - `Main`, 20
- `app`
 - `Main`, 21
- `cnn_model`
 - `ai_analysis`, 19
- `criterion`
 - `train_model`, 23
- `device`
 - `ai_analysis`, 19
 - `train_model`, 23
- `draw_matches`
 - `traditional_analysis`, 21
- `EPOCHS`
 - `train_model`, 23
- `fc`
 - `ai_analysis`, 19
 - `ai_analysis.ForgeryLSTM`, 29
 - `train_model`, 23
- `find_forgery_matches`
 - `traditional_analysis`, 21
- `forgery-ui Directory Reference`, 17
- `forgery-ui/README.md`, 31
- `forward`
 - `ai_analysis.ForgeryLSTM`, 28
- `get_detector`
 - `traditional_analysis`, 22
- Görüntü Sahteciliği Tespit Sistemi API Dökümantasyonu, 1
- `host`
 - `Main`, 21
- `images`
 - `train_model`, 24
- `labels`
 - `train_model`, 24
- `loss`
 - `train_model`, 24
- `lstm`
 - `ai_analysis.ForgeryLSTM`, 29
- `lstm_model`
 - `ai_analysis`, 20
- `Main`, 20
 - `allow_headers`, 20
 - `allow_methods`, 20
 - `allow_origins`, 20
 - `analyze_image`, 20
 - `app`, 21
 - `host`, 21
 - `port`, 21
- `Main.py`, 31
- `model`
 - `train_model`, 24
- `model_path`
 - `ai_analysis`, 20
- `optimizer`
 - `train_model`, 24
- `outputs`
 - `train_model`, 24
- `port`
 - `Main`, 21
- `Readme.md`, 31
- `resnet`
 - `ai_analysis`, 20
- `run_ai_analysis`
 - `ai_analysis`, 19
- `run_traditional_analysis`

- traditional_analysis, [22](#)
- running_loss
 - train_model, [24](#)
- test_dataset
 - train_model, [24](#)
- test_loader
 - train_model, [24](#)
- traditional_analysis, [21](#)
 - draw_matches, [21](#)
 - find_forgery_matches, [21](#)
 - get_detector, [22](#)
 - run_traditional_analysis, [22](#)
- traditional_analysis.py, [32](#)
- train_dataset
 - train_model, [24](#)
- train_loader
 - train_model, [24](#)
- train_model, [23](#)
 - criterion, [23](#)
 - device, [23](#)
 - EPOCHS, [23](#)
 - fc, [23](#)
 - images, [24](#)
 - labels, [24](#)
 - loss, [24](#)
 - model, [24](#)
 - optimizer, [24](#)
 - outputs, [24](#)
 - running_loss, [24](#)
 - test_dataset, [24](#)
 - test_loader, [24](#)
 - train_dataset, [24](#)
 - train_loader, [24](#)
 - transform, [24](#)
- train_model.py, [32](#)
- transform
 - ai_analysis, [20](#)
 - train_model, [24](#)

Görüntü Sahteciliği Tespit Sistemi (Image Forgery Detection), [3](#)